

Yesterday 3:26 PM

What is reflection?
How can you get a class using reflection?
How can you access private fields?
How can you invoke a method dynamically?
Where is reflection used?
How does Spring use reflection?
How does dependency injection work with reflection?
What are disadvantages of reflection?
Why can reflection impact performance?
What is the difference between compile-time and runtime discovery?

For a **5-year Java/Spring Boot interview**, you should be able to explain Reflection not just as a definition, but also **how Spring uses it internally**.

1. What is Reflection?

Reflection is a Java mechanism that allows us to inspect and manipulate classes, fields, methods, constructors, and annotations at runtime.

Normally, Java code knows the class structure at compile time. With reflection, we can discover that structure **while the application is running**.

Example:

 Java



```
Class<?> clazz = Employee.class;

System.out.println(clazz.getName());

for (var method : clazz.getDeclaredMethods()) {
    System.out.println(method.getName());
}
```

Here, we are discovering methods dynamically at runtime.

Interview answer:

"Reflection allows Java applications to inspect and interact with classes, methods, fields, constructors, and annotations at runtime, even when some information wasn't explicitly known at compile time."

2. How can you get a Class using Reflection?

There are three common ways:

Using `.class`

</> Java



```
Class<?> clazz = Employee.class;
```

Using an object

</> Java



```
Employee employee = new Employee();  
  
Class<?> clazz = employee.getClass();
```

Using class name

</> Java



```
Class<?> clazz = Class.forName("com.example.Employee");
```

The third approach is especially important because the class can be identified dynamically.

For example:

</> Java



```
String className = "com.example.Employee";  
  
Class<?> clazz = Class.forName(className);
```

Interview point

`Class.forName()` can throw:

</> Java



`ClassNotFoundException`

because the class is loaded dynamically.

3. How can you access private fields?

Reflection allows you to access a private field.

Suppose:

</> Java



```
class Employee {  
  
    private String name = "Manoj";  
}
```

We can do:

</> Java



```
Employee employee = new Employee();  
  
Field field = Employee.class.getDeclaredField("name");  
  
field.setAccessible(true);  
  
String name = (String) field.get(employee);  
  
System.out.println(name);
```

Important distinction

</> Java



`getField()`

returns **public fields**, including inherited public fields.

</> Java



```
getDeclaredField()
```

returns fields declared directly inside the class, including **private fields**.

Similarly:

</> Java



```
getDeclaredMethods()  
getDeclaredConstructors()
```

can discover non-public members.

Modern Java note

`setAccessible(true)` can be restricted by Java's module system in some situations. In modern Java, strong encapsulation can prevent reflective access to certain JDK/internal classes.

4. How can you invoke a method dynamically?

Suppose:

</> Java



```
class Employee {  
  
    public void printName() {  
        System.out.println("Manoj");  
    }  
}
```

Using reflection:

</> Java



```
Employee employee = new Employee();  
  
Method method =
```

```
Employee.class.getDeclaredMethod("printName");

method.invoke(employee);
```

Output:

Manoj



You can also invoke methods with parameters:

```
</> Java

Method method =
    Employee.class.getDeclaredMethod("setName", String.class);

method.invoke(employee, "Manoj");
```



So the method doesn't have to be directly called like:

```
</> Java

employee.setName("Manoj");
```



The method name and parameters can be discovered dynamically.

5. Where is Reflection used?

Reflection is heavily used by **frameworks and infrastructure libraries**.

Common examples:

- Spring / Spring Boot
- Hibernate / JPA
- JUnit
- Jackson
- Dependency Injection frameworks
- Serialization/deserialization frameworks
- Testing frameworks
- Plugin systems
- Annotation processing at runtime

For example, Jackson can inspect a class and discover:

</> Java



```
class Employee {  
  
    private String name;  
    private int age;  
  
}
```

and use reflection/introspection to map JSON properties to Java fields or setters.

6. How does Spring use Reflection?

This is a **very important** interview question.

Consider:

</> Java



```
@Service  
public class OrderService {  
  
    @Autowired  
    private PaymentService paymentService;  
  
}
```

You don't explicitly create:

</> Java



```
new PaymentService();
```

Spring does it for you.

Conceptually, Spring:

Step 1 — Scans classes

Spring scans configured packages looking for classes such as:

</> Java



```
@Component  
@Service
```

```
@Repository  
@Controller
```

Step 2 — Reads metadata

Spring identifies:

```
</> Java  
  
@Service  
public class OrderService
```

and creates a bean definition.

Step 3 — Creates the object

Conceptually:

```
</> Java  
  
Class<?> clazz = OrderService.class;  
  
Object object = clazz.getDeclaredConstructor()  
                .newInstance();
```

Step 4 — Performs dependency injection

Spring discovers the dependency:

```
</> Java  
  
@Autowired  
private PaymentService paymentService;
```

Conceptually:

```
</> Java  
  
Field field =  
    clazz.getDeclaredField("paymentService");  
  
field.setAccessible(true);  
  
field.set(orderServiceObject, paymentServiceObject);
```

This is simplified, but it gives you the correct mental model.

Step 5 — Calls lifecycle methods

Spring can also inspect annotations and invoke methods during bean initialization.

For example:

</> Java



```
@PostConstruct
public void init() {
    // initialization
}
```

7. How does Dependency Injection work with Reflection?

Suppose we have:

</> Java



```
@Service
class OrderService {

    @Autowired
    private PaymentService paymentService;
}
```

Spring's container maintains objects such as:

ApplicationContext



```
|
+---- OrderService
|
+---- PaymentService
```

When Spring creates `OrderService`, it detects:

</> Java



```
@Autowired
```

and determines that `PaymentService` is required.

Conceptually:

</> Java



```
Field field = OrderService.class
    .getDeclaredField("paymentService");

field.setAccessible(true);

field.set(orderService, paymentService);
```

So:

@Autowired



```
↓
Spring scans metadata
↓
Finds dependency
↓
Gets dependency from IoC container
↓
Reflection accesses injection point
↓
Dependency assigned
```

Important interview nuance

Don't say:

"Spring uses reflection for everything."

A better answer is:

"Spring uses reflection and other mechanisms such as generated proxies, bytecode/code generation, and framework metadata to implement features like dependency injection, annotation processing, bean creation, and method invocation."

That's a much stronger **5-year experience answer**.

8. What are disadvantages of Reflection?

There are several.

1. Performance overhead

Reflection is generally slower than direct method/field access.

Direct:

```
</> Java  
employee.getName();
```



Reflection:

```
</> Java  
method.invoke(employee);
```



The JVM has more work to do.

2. Breaks encapsulation

Normally:

```
</> Java  
private String name;
```



means outside code shouldn't directly access it.

Reflection can potentially bypass those access restrictions.

3. Less compile-time safety

Normal code:

```
</> Java  
employee.getName();
```



If `getName()` doesn't exist, the compiler tells you.

Reflection:

```
</> Java  
clazz.getDeclaredMethod("getName");
```



The problem may only appear **at runtime**.

For example:

```
NoSuchMethodException  
IllegalAccessException  
InvocationTargetException
```



4. Code becomes harder to understand

Reflection-heavy code is often harder to:

- debug
- refactor
- maintain
- understand

5. Can have issues with Java modules

Modern Java's module system provides stronger encapsulation, so some reflective access can be restricted.

9. Why can Reflection impact performance?

This is a common follow-up.

Consider direct invocation:

```
</> Java  
  
employee.calculateSalary();
```



The JVM knows the target method through normal bytecode and can perform optimizations such as:

- JIT compilation
- method inlining
- other runtime optimizations

With:

</> Java



```
method.invoke(employee);
```

the method is determined dynamically.

There is additional work around:

Method lookup



Access checks



Argument handling



Dynamic invocation



Result handling



Therefore, reflective calls can have more overhead than normal direct calls.

Important practical point

Don't say:

"Reflection is always slow."

Better:

"Reflection generally has higher overhead than direct access, but in most framework initialization or configuration scenarios this overhead is acceptable. It becomes more significant when reflection is performed repeatedly inside a hot, high-throughput path."

That's a **strong interview answer**.

10. Compile-time vs Runtime Discovery

This is an important conceptual difference.

Compile-time discovery

The compiler knows the class/method while compiling.

Example:

</> Java



```
Employee employee = new Employee();  
  
employee.getName();
```

The compiler checks:

Does Employee exist?
Does getName() exist?
Are parameters correct?
Is the method accessible?



If something is wrong, compilation fails.

Runtime discovery

The application discovers information while running.

Example:

</> Java



```
String className = "com.example.Employee";  
  
Class<?> clazz = Class.forName(className);  
  
Method method =  
    clazz.getDeclaredMethod("getName");  
  
method.invoke(employee);
```

The compiler doesn't necessarily know which class/method will be selected.

The discovery happens at runtime.

Compile-time vs Runtime

Compile Time	Runtime
Known by compiler	Discovered while application runs

Compile Time	Runtime
Strong type checking	Less compile-time safety
Usually faster	Usually more overhead
Easier refactoring	Can be harder to refactor
Errors caught earlier	Errors may appear at runtime
<code>employee.getName()</code>	<code>method.invoke(employee)</code>

★ Best 5-Year Interview Answer

If the interviewer asks "Explain Reflection and its use in Spring", you can answer:

"Reflection is a Java mechanism that allows us to inspect and manipulate classes, fields, methods, constructors, and annotations at runtime. We can obtain a Class object using `ClassName.class` , `object.getClass()` , or `Class.forName()` . Using reflection, we can dynamically access fields and invoke methods.

Frameworks like Spring use reflection extensively for component scanning, reading annotations, creating beans, and performing dependency injection. For example, when Spring sees an `@Autowired` field, it can identify the field through metadata/reflection and inject the corresponding bean from the IoC container.

The disadvantages are reduced compile-time safety, harder debugging and maintenance, possible encapsulation issues, and performance overhead compared with direct method or field access. However, the overhead is usually acceptable for framework-level operations such as startup and dependency injection, rather than repeatedly using reflection in performance-critical business logic."

One line to remember

Reflection = "Discover and interact with Java's structure at runtime."



What is an annotation?

Built-in vs custom annotations.

@Retention

@Target

@Documented

@Inherited

Runtime vs compile-time annotations.

How would you create a custom annotation?

How does Spring process annotations?

How would you create an annotation-based validation mechanism

Show more ▾

Absolutely. For a **5-year Java interview**, these topics are important because interviewers often go beyond definitions and ask "why?" + "how does Java/Spring implement it?"

Part 1 — Annotations

1. What is an Annotation?

An **annotation** is **metadata attached to Java code** such as a class, method, field, parameter, or constructor.

It provides additional information to:

- Compiler
- JVM
- Frameworks
- Tools

Example:

</> Java



```
@Override
public String toString() {
    return "Employee";
}
```

Here `@Override` tells the compiler that this method is intended to override a parent method.

Spring examples:

</> Java



```
@Service
public class OrderService {
}
```

</> Java



```
@Autowired
private PaymentService paymentService;
```

</> Java



```
@Transactional
public void createOrder() {
}
```

The annotation itself generally doesn't perform the behavior. **Some tool/framework reads the annotation and performs the behavior.**

2. Built-in vs Custom Annotations

Built-in annotations

Java provides annotations such as:

</> Java



```
@Override
@Deprecated
@SuppressWarnings
```

Example:

</> Java



```
@Override  
public void run() {  
}
```

Custom annotations

We can create our own annotation.

Example:

```
</> Java  
  
@Retention(RetentionPolicy.RUNTIME)  
@Target(ElementType.METHOD)  
public @interface Auditable {  
}
```

Usage:

```
</> Java  
  
@Auditable  
public void transferMoney() {  
}
```

A framework or our own code can inspect `@Auditable` and perform some operation.

3. What is @Retention ?

`@Retention` specifies how long the annotation should be available.

There are three values:

```
</> Java  
  
RetentionPolicy.SOURCE  
RetentionPolicy.CLASS  
RetentionPolicy.RUNTIME
```

SOURCE

Available only in source code.

```
</> Java  
  
@Retention(RetentionPolicy.SOURCE)
```



The annotation is discarded during compilation.

Example:

```
</> Java  
  
@Override
```



The compiler uses it, but it isn't needed at runtime.

CLASS

Stored in `.class` files but not necessarily available through runtime reflection.

```
</> Java  
  
@Retention(RetentionPolicy.CLASS)
```



This is the default retention policy.

RUNTIME

Available at runtime through reflection.

```
</> Java  
  
@Retention(RetentionPolicy.RUNTIME)
```



This is extremely important for frameworks.

For example, Spring needs to inspect annotations such as:

```
</> Java
```



```
@Component  
@Service  
@Autowired
```

at runtime.

Interview shortcut

```
SOURCE → source code  
CLASS → .class file  
RUNTIME → reflection/runtime
```



4. What is @Target ?

@Target specifies where an annotation can be applied.

Example:

```
</> Java  
  
@Target(ElementType.METHOD)
```



means:

```
</> Java  
  
@MyAnnotation  
public void test() {  
}
```



but you can't apply it to a class.

Common targets:

```
</> Java  
  
ElementType.TYPE  
ElementType.METHOD  
ElementType.FIELD  
ElementType.PARAMETER  
ElementType.CONSTRUCTOR  
ElementType.LOCAL_VARIABLE
```



```
ElementType.ANNOTATION_TYPE  
ElementType.PACKAGE
```

You can specify multiple:

```
</> Java  
  
@Target({  
    ElementType.TYPE,  
    ElementType.METHOD  
})
```



5. What is @Documented ?

@Documented indicates that the annotation should appear in generated JavaDoc.

Example:

```
</> Java  
  
@Documented  
public @interface Auditable {  
}
```



Without it, the annotation doesn't automatically appear in generated documentation.

Interview answer

" @Documented tells the JavaDoc tool to include the annotation in generated API documentation."

It's less important than @Retention and @Target , but know the definition.

6. What is @Inherited ?

@Inherited allows a **class-level annotation** to be inherited by subclasses.

Example:

```
</> Java
```



```
@Inherited
@Retention(RetentionPolicy.RUNTIME)
@Target(ElementType.TYPE)
public @interface Auditable {
}
```

Then:

```
</> Java

@Auditable
class Parent {
}

class Child extends Parent {
}
```

Child can see the inherited annotation through:

```
</> Java

Child.class.isAnnotationPresent(Auditable.class);
```

Important interview trap

@Inherited applies to **class annotations only**.

It does **not** make:

- method annotations
- field annotations
- parameter annotations

automatically inherited.

7. Runtime vs Compile-Time Annotations

The key question is:

Who needs the annotation?

Compile-time

Used by the compiler.

Example:

</> Java

@Override

The compiler checks whether you're actually overriding a parent method.

Runtime

Used by applications/frameworks while running.

Example:

</> Java

@Service

@Transactional

@Autowired

Spring can inspect these annotations during application startup/runtime.

Simple comparison

Compile-time	Runtime
Compiler processes it	Application/framework processes it
Useful for compiler checks	Useful for runtime behavior
Example @Override	Example Spring annotations
Usually SOURCE retention	Usually RUNTIME retention

Strictly speaking, **retention policy and when an annotation is processed are related but not identical concepts**. An annotation can be stored in class files without being runtime-reflectable.

8. How would you create a Custom Annotation?

Example:

</> Java



```
@Retention(RetentionPolicy.RUNTIME)
@Target(ElementType.METHOD)
public @interface Auditable {
    String action();
}
```

Use it:

</> Java



```
@Auditable(action = "CREATE_ORDER")
public void createOrder() {
}
```

Read it:

</> Java



```
Method method = OrderService.class
    .getDeclaredMethod("createOrder");

Auditable annotation =
    method.getAnnotation(Auditable.class);

System.out.println(annotation.action());
```

Output:

CREATE_ORDER



So the complete flow is:

```
Create annotation
    ↓
Apply annotation
    ↓
Retain at runtime
    ↓
Read using reflection
    ↓
Perform some behavior
```



9. How does Spring process annotations?

This is a very important 5-year interview question.

Suppose:

`</> Java`



```
@Service
public class PaymentService {
}
```

Spring doesn't magically understand `@Service`.

Spring has mechanisms that:

1. Scan classes/packages
2. Detect annotations
3. Create bean definitions
4. Instantiate beans
5. Resolve dependencies
6. Apply post-processors/proxies where needed

Conceptually:

`@ComponentScan`



Find `PaymentService`



Detect `@Service`



Create `BeanDefinition`



Create `PaymentService` bean



Put bean into `ApplicationContext`



For:

`</> Java`



```
@Autowired
private PaymentService paymentService;
```

Spring identifies the injection point and resolves the corresponding bean.

For:

</> Java



```
@Transactional
public void transfer() {
}
```

Spring can create a proxy/interceptor around the bean so transaction behavior happens around the method invocation.

Strong interview answer

"Spring uses annotation metadata together with mechanisms such as component scanning, bean post-processors, reflection, and proxies to implement annotation-driven behavior. Different annotations are processed by different parts of the Spring container."

That is better than simply saying:

"Spring uses reflection."

10. How would you create an annotation-based validation mechanism?

This is a nice **design/interview** question.

Suppose we want:

</> Java



```
public class User {

    @NotBlank
    private String name;

    @Positive
    private int age;
}
```

We can create:

</> Java



```
@Retention(RetentionPolicy.RUNTIME)
@Target(ElementType.FIELD)
public @interface NotBlank {
}
```

Then:

</> Java



```
@Retention(RetentionPolicy.RUNTIME)
@Target(ElementType.FIELD)
public @interface Positive {
}
```

Now create a validator:

</> Java



```
public class Validator {

    public static void validate(Object object)
        throws IllegalAccessException {

        for (Field field : object.getClass().getDeclaredFields()) {

            field.setAccessible(true);

            if (field.isAnnotationPresent(NotBlank.class)) {

                Object value = field.get(object);

                if (value == null ||
                    value.toString().isBlank()) {

                    throw new IllegalArgumentException(
                        field.getName() + " cannot be blank"
                    );
                }
            }

            if (field.isAnnotationPresent(Positive.class)) {

                Object value = field.get(object);

                if (value instanceof Number number &&
```

```

        number.doubleValue() <= 0) {

        throw new IllegalArgumentException(
            field.getName() + " must be positive"
        );
    }
}
}
}
}
}
}
}

```

Usage:

</> **Java**



```

User user = new User();

Validator.validate(user);

```

Production-level design

For a real framework, I would avoid putting every validation rule into one giant validator.

I'd design:

```

Annotation
  ↓
Validator interface
  ↓
NotBlankValidator
PositiveValidator
EmailValidator
...
  ↓
ValidationEngine

```



For example:

</> **Java**



```

interface FieldValidator<A extends Annotation> {

    void validate(
        Object value,
        A annotation
    );
}

```

```
    );  
}
```

This makes adding a new annotation easier.

Part 2 — Generics

11. Why were Generics introduced?

Generics were introduced primarily for:

1. Type safety

Without generics:

```
</> Java  
  
List list = new ArrayList();  
  
list.add("Java");  
list.add(10);  
  
String value = (String) list.get(1);
```

This fails at runtime.

With generics:

```
</> Java  
  
List<String> list = new ArrayList<>();  
  
list.add("Java");
```

This is prevented:

```
</> Java  
  
list.add(10); // compile-time error
```

2. Reduce casting

Without:

</> Java



```
String value = (String) list.get(0);
```

With:

</> Java



```
String value = list.get(0);
```

3. Reusable code

Instead of:

</> Java



```
StringBox  
IntegerBox  
EmployeeBox
```

we can create:

</> Java



```
class Box<T> {  
    private T value;  
}
```

and use:

</> Java



```
Box<String>  
Box<Integer>  
Box<Employee>
```

12. Generic Class vs Generic Method

Generic class

The type parameter belongs to the class.

</> Java



```
class Box<T> {  
  
    private T value;  
  
    public T getValue() {  
        return value;  
    }  
}
```

Usage:

</> Java



```
Box<String> box = new Box<>();
```

Here τ belongs to the class.

Generic method

The type parameter belongs only to the method.

</> Java



```
public <T> T getValue(T value) {  
    return value;  
}
```

Example:

</> Java



```
String result = getValue("Java");  
  
Integer number = getValue(10);
```

Important syntax

```
</> Java
```



```
public <T> T method(T value)
    ↑
    declares T
```

The `<T>` before the return type declares the method-level generic parameter.

13. What is Type Erasure?

Type erasure means Java removes most generic type information during compilation and uses the erased type in the generated bytecode.

For example:

```
</> Java
```



```
List<String>
```

and:

```
</> Java
```



```
List<Integer>
```

both essentially become:

```
</> Java
```



```
List
```

at runtime.

Consider:

```
</> Java
```



```
List<String> names = new ArrayList<>();
```

The JVM generally doesn't retain `String` as the runtime type parameter of that `List` object.

The compiler performs necessary type checks and inserts casts where needed.

14. Why does Java use Type Erasure?

The major reason is **backward compatibility**.

Generics were introduced in Java 5.

There were already huge amounts of Java code using:

```
</> Java  
  
List  
Map  
Collection
```



Java wanted generic code to work with existing non-generic bytecode/libraries.

Therefore:

```
</> Java  
  
List<String>
```



could interact with older:

```
</> Java  
  
List
```



APIs.

Interview answer

"Java uses type erasure primarily to maintain backward compatibility with pre-generics Java bytecode and APIs."

15. What is a Bounded Type Parameter?

A bounded type parameter restricts what types can be used.

Example:

```
</> Java
<T extends Number>
```



means:

T must be Number or a subclass of Number .

Example:

```
</> Java
public <T extends Number> double doubleValue(T value) {
    return value.doubleValue();
}
```



Valid:

```
</> Java
doubleValue(10);
doubleValue(10.5);
```



because:

```
Integer extends Number
Double extends Number
```



Invalid:

```
</> Java
doubleValue("Java");
```



16. <T extends Number> VS <? extends Number>

This is a **very common** interview question.

<T extends Number>

This declares a **type parameter**.

```
<> Java
public <T extends Number> void process(T value)
```

T is a named type that can be used throughout the method signature/body.

```
<? extends Number>
```

This is a **wildcard**.

```
<> Java
List<? extends Number>
```

It means:

A List of some unknown type that extends Number.

For example:

```
<> Java
List<Integer>
List<Double>
List<Float>
```

can all be assigned to:

```
<> Java
List<? extends Number>
```

17. List<? extends Number>

Suppose:

```
<> Java
```

```
List<Integer> integers = new ArrayList<>();

List<? extends Number> numbers = integers;
```

You can safely read:

```
</> Java

Number n = numbers.get(0);
```

because whatever the actual type is, it is guaranteed to be some subtype of `Number`.

But you cannot safely add:

```
</> Java

numbers.add(10); // ❌
```

Why?

Because the actual list might be:

```
</> Java

List<Double>
```

Adding an `Integer` would be unsafe.

Rule

```
? extends → producer → READ
```

18. List<? super Integer>

Now:

```
</> Java

List<Number> numbers = new ArrayList<>();

List<? super Integer> list = numbers;
```

It could represent:

</> Java



```
List<Integer>
List<Number>
List<Object>
```

You can safely add:

</> Java



```
list.add(10);
```

because `Integer` can be stored in all of those.

But reading is less specific:

</> Java



```
Object value = list.get(0);
```

You can safely assume only `Object` .

Rule

? super → consumer → WRITE



19. PECS Principle

PECS = Producer Extends, Consumer Super

Producer → extends

If you're mainly **reading** values:

</> Java



```
List<? extends Number>
```

The collection produces `Number` values.

Consumer → `super`

If you're mainly **adding** values:

```
</> Java
```

```
List<? super Integer>
```



The collection consumes `Integer` values.

Easy memory trick

```
EXTENDS → GET
```

```
SUPER → PUT
```



Not absolutely every operation follows this simplified rule, but it's the correct design guideline for collection APIs.

20. Why can't we do `new T()` ?

Suppose:

```
</> Java
```

```
class Factory<T> {

    public T create() {
        return new T(); // ✗
    }

}
```



The problem is **type erasure**.

At runtime, the JVM doesn't generally know what `T` actually represents.

For:

```
</> Java
```

```
Factory<String>
```



it could be `String`.

For:

```
</> Java
```



```
Factory<Employee>
```

it could be `Employee` .

The JVM cannot simply execute:

```
</> Java
```



```
new T()
```

because the concrete class isn't available that way.

Common solution

Pass a `Class<T>` :

```
</> Java
```



```
class Factory<T> {  
  
    private final Class<T> type;  
  
    Factory(Class<T> type) {  
        this.type = type;  
    }  
  
    T create() throws Exception {  
        return type.getDeclaredConstructor().newInstance();  
    }  
}
```

Usage:

```
</> Java
```



```
Factory<Employee> factory =  
    new Factory<>(Employee.class);
```

21. Why can't we create Generic Arrays?

This is prohibited:

</> Java



```
T[] array = new T[10]; // ❌
```

Because arrays know their component type at runtime, while generic type information is erased.

Imagine:

</> Java



```
List<String>[] array = new List<String>[10];
```

If this were allowed, Java could potentially violate type safety through array covariance and raw types.

Instead, you often see:

</> Java



```
@SuppressWarnings("unchecked")
T[] array = (T[]) new Object[10];
```

But this requires care because the runtime array is actually an `Object[]`.

Interview answer

"Generic arrays are prohibited because arrays are reified at runtime while generic type parameters are erased, which can create type-safety problems."

22. Can static methods use class-level generic type?

Suppose:

</> Java



```
class Box<T> {

    static T getValue() { // ❌
        return null;
    }
}
```

```
}
}
```

No.

Why?

Because T belongs to the **instance/class parameterization**.

These are different:

```
</> Java
Box<String>
Box<Integer>
```

But static members belong to the class itself, not a particular parameterized instance.

So what should the static method do?

Declare its own type parameter:

```
</> Java
class Box<T> {
    public static <T> T create(T value) {
        return value;
    }
}
```

Notice:

```
</> Java
static <T> T
      ↑
method-level T
```

This T is independent of the class-level T .

23. Raw Type vs Parameterized Type

Raw type

</> Java



```
List list = new ArrayList();
```

No generic type information.

Problems:

</> Java



```
list.add("Java");  
list.add(10);
```

You can put different types into it.

Parameterized type

</> Java



```
List<String> list = new ArrayList<>();
```

Now Java provides compile-time type safety.

</> Java



```
list.add("Java");  
  
list.add(10); // ❌
```

Interview answer

"Raw types are legacy usage of generic classes without specifying type parameters. They sacrifice compile-time type safety and can produce unchecked warnings."

24. What problems occur because of Type Erasure?

This is a good advanced question.

1. Cannot do:

```
</> Java  
  
new T();
```



2. Cannot create:

```
</> Java  
  
new T[10];
```



3. Cannot use instanceof like:

```
</> Java  
  
if (obj instanceof List<String>) { // ✗  
}
```



Instead:

```
</> Java  
  
if (obj instanceof List<?>) {  
}
```



4. Cannot overload methods only by generic parameter

This doesn't work:

```
</> Java  
  
void process(List<String> list) {}  
  
void process(List<Integer> list) {} // ✗
```



After erasure both look like:

```
</> Java
```



```
process(List)
```

so their signatures collide.

5. Runtime doesn't normally know the generic type

For example:

```
</> Java
```



```
List<String>
```

and:

```
</> Java
```



```
List<Integer>
```

have the same runtime class:

```
</> Java
```



```
ArrayList
```

★ Most Important Interview Summary

Remember this table:

Concept	Key point
Annotation	Metadata attached to Java code
@Retention	How long annotation exists
SOURCE	Source code only
CLASS	Stored in .class

Concept	Key point
RUNTIME	Available through reflection
@Target	Where annotation can be used
@Documented	Included in JavaDoc
@Inherited	Class-level annotation inheritance
Generics	Compile-time type safety
Type erasure	Generic type mostly removed at runtime
<code><T extends Number></code>	Named bounded type parameter
<code>? extends Number</code>	Unknown subtype of Number
<code>? super Integer</code>	Unknown supertype of Integer
PECS	Producer Extends, Consumer Super
<code>new T()</code>	Not allowed because of type erasure
Generic array	Not directly allowed because of erasure + runtime array typing
Static + class T	Not allowed; static method needs its own <code><T></code>
Raw type	Generic type used without type parameter

★ One-liners to memorize

Annotations:

"Annotations provide metadata; frameworks or tools decide what behavior that metadata triggers."

Reflection:

"Reflection allows us to inspect and interact with Java types at runtime."

Generics:

"Generics provide compile-time type safety and reusable type-independent code."

Type erasure:

"Java implements generics largely through type erasure to preserve backward compatibility."

PECS:

Producer Extends, Consumer Super.



ChatGPT can make mistakes. Check important info.